

Vision Aided Navigation - Exercise 1 Report

Nir Ellor Waizner, Ayala Hourl

- 1.1.** Using the ORB (FAST) feature detection algorithm, we identified and extracted over 500 key points from each image in the initial stereo pair. ORB was selected for its computational efficiency and robustness against rotation, making it an ideal choice for real-time stereo matching tasks.



From the visualized results, we can infer that the detector successfully identifies high-contrast corners and textures, particularly along building edges and foliage. The consistent distribution of green key-points across both frames suggests that the features are repeatable and suitable for stereo correspondence.

- 1.2.** Using the ORB (BRIEF) feature detection algorithm, which complements FAST of ORB, the descriptors of the two first features:

Image 1 Descriptor 0:

[35 181 144 26 253 157 176 239 219 49 56 28 174 184 76 148 131 75 194 16 23 106 218 243 47 92 20 123 218 12 203 143]

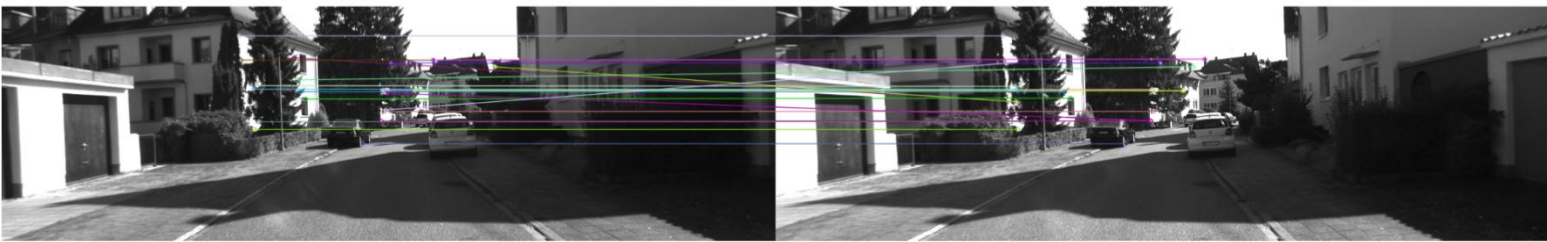
Image 1 Descriptor 1:

[120 184 234 56 105 217 80 71 227 93 164 41 35 149 66 26 128 186 236 16 14 107 226 17 211 241 16 123 98 132 78 90]

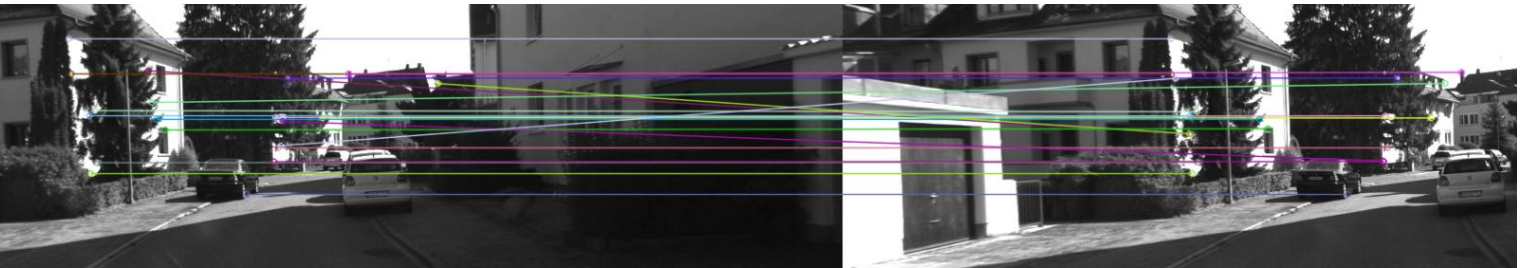
These 32-element vectors represent 256-bit binary strings, where each value is the decimal equivalent of an 8-bit segment. The distinct numerical differences between Descriptor 0 and Descriptor 1 indicate that the algorithm effectively captures unique local intensity patterns, which will allow the matching algorithm to differentiate between disparate points in the scene.

1.3. We matched descriptors using BFMatcher with Hamming distance.

20 Random Matches



For better visualisation:



We can observe that some of the matches are incorrect. To improve the matching quality and remove ambiguous matches, we apply the ratio test in section 1.4.

1.4. In this section, we applied Lowe's ratio test to filter out unreliable matches.

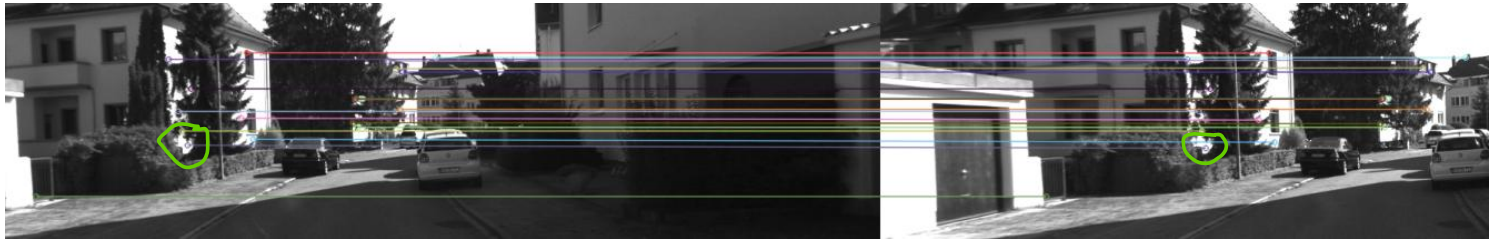
For each descriptor, we retrieved the two nearest neighbors using KNN matching.

A match was accepted only if the distance of the best match was significantly smaller than the second-best match, i.e., if the ratio between them was below a chosen threshold (we used a ratio value of 0.75).

Formally, we applied the condition: $\text{best}/\text{second} < \text{ratio}$, which is equivalent to $m.\text{distance} < 0.75 * n.\text{distance}$.

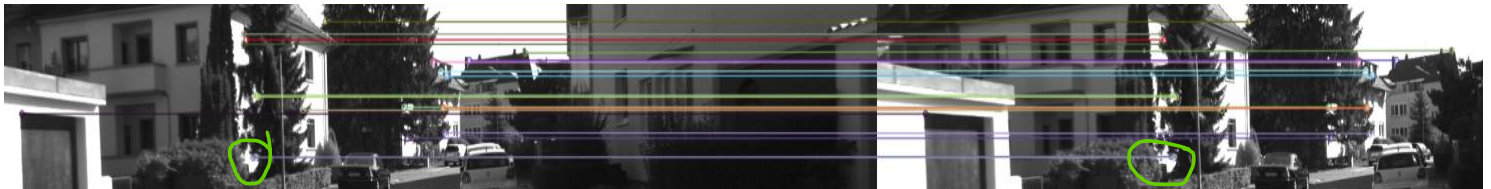
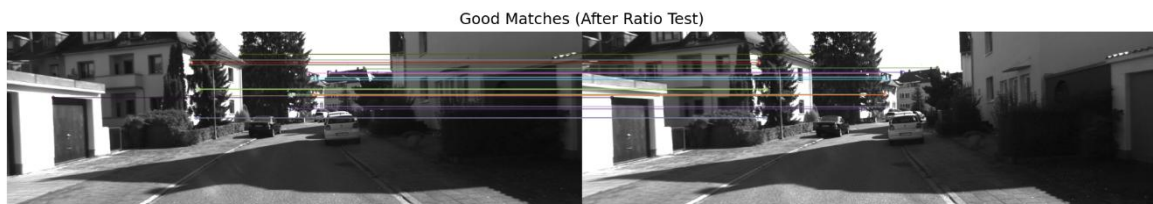
While the literature (e.g., Wikipedia) often uses a threshold around 0.8, we chose a stricter value of 0.75 to be more selective and reduce ambiguous matches. This ensures that the best match is distinct and not ambiguous compared to other candidates.

As a result, ambiguous matches are rejected, improving the overall reliability and quality of the matching.



Upon inspecting the matching results, we observed that some correspondences are not perfectly aligned. Although they fall within the same general area, they do not match the exact corresponding point, indicating residual ambiguity.

To address this, we further tightened the ratio threshold to make the filtering more restrictive.



It can be observed that the results have improved. So the chosen ratio is: 0.6.

How many matches were discarded?

Total matches found: 500

Good matches: 177

Rejected matches: 323



We now present a correct match that was rejected by the significance (ratio) test.

It can be observed that the corresponding points are well aligned in both images, indicating a correct match.

However, this match was discarded due to the strict ratio threshold, which rejected it as ambiguous.

From this, we learn that the choice of the ratio threshold involves a trade-off: a stricter ratio (e.g., 0.75 or lower) reduces ambiguous matches but may also discard some correct ones, while a higher ratio preserves more matches at the cost of increased ambiguity.

Functions:

draw_random_matches:

This function randomly selects a subset of matches and visualizes them by drawing lines between corresponding keypoints in the left and right images. It is used to present a representative sample of matches.

ratio_test_match:

This function performs KNN matching ($k=2$) between descriptors and applies Lowe's ratio test to filter matches. It returns two lists: reliable matches (`good_matches`) and rejected matches based on the ratio threshold.

draw_matches:

This function visualizes a given set of matches (up to a specified number) by drawing lines between corresponding keypoints in the two images. It is used to display either accepted or rejected matches.

show_multiple_rejected:

This function selects a few rejected matches and displays their corresponding keypoints as circles on both images (without connecting lines). It is used to highlight matches that were rejected by the ratio test for analysis.

detect_orb:

This function serves as the core processing unit for feature detection. It initializes the ORB detector, identifies keypoints, and computes their corresponding binary descriptors. It is configured to extract a specific number of features to ensure a dense representation of the scene.

visualize_side_by_side:

A helper utility designed for comparative analysis. It plots two images in a single figure using subplots, allowing for an immediate visual comparison of original frames, keypoint distributions, or processed results.

match_descriptors:

This function establishes the initial correspondence between the stereo pair. It uses a Brute-Force matcher with Hamming distance to find the closest descriptor in the right image for every descriptor in the left image, representing the "raw" matching phase before any filtering.

get_orb_features:

A wrapper function that encapsulates the ORB detector and descriptor extractor. It simplifies the main execution flow by returning both the spatial locations (keypoints) and the visual fingerprints (descriptors) in a single call.

plot_stereo_side_by_side :

An optimized version of the visualization utility specifically tailored for stereo pairs. It handles grayscale mapping and formatting to ensure that the left and right perspectives are displayed clearly for report-ready figures.

draw_matches_custom:

A robust visualization tool that combines random sampling and OpenCV's drawing utilities. It prevents visual clutter by limiting the number of displayed matches and uses specific flags to highlight only the connected pairs, making it easier to evaluate matching accuracy.